

PIC16F877A Fan Controller Code Explanation

Project idea: This project controls the speed and direction of a DC fan or motor using the PIC16F877A microcontroller. A potentiometer controls speed through ADC and PWM, one push button starts or stops the motor, another push button changes direction, an LCD shows the current status, LEDs indicate the speed range, and a buzzer warns when the speed is near maximum.

1. Configuration Bits

```
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config BOREN = ON
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config WRT = OFF
#pragma config CP = OFF
```

These lines configure the PIC16F877A before the main program starts. `FOSC = HS` selects the high-speed external oscillator mode. The code later defines the clock as 4 MHz, so the real hardware oscillator should match that value for accurate delays.

`WDTE = OFF` disables the watchdog timer, so the microcontroller will not reset automatically if the program takes too long in a loop. `PWRTE = ON` enables the power-up timer, giving the chip a short stabilization delay after power is applied. `BOREN = ON` enables brown-out reset, which helps reset the chip if the supply voltage drops too low.

`LVP = OFF` disables low-voltage programming, which prevents programming mode from interfering with normal I/O operation. `CPD`, `WRT`, and `CP` disable EEPROM protection, flash write protection, and code protection, making the project easier to program and debug.

2. Header File and Clock Frequency

```
#include <xc.h>

#define _XTAL_FREQ 4000000
```

`xc.h` is the main XC8 header file. It provides register names, bit names, and device-specific definitions for the PIC microcontroller. `_XTAL_FREQ` tells the compiler the oscillator frequency. This is required by the delay functions `__delay_ms()` and `__delay_us()`.

3. Motor and Button Pin Definitions

```
#define IN1 RD0
#define IN2 RD1
```

```
#define START_STOP_BTN RB0
#define DIR_BTN RB1
```

These macros give meaningful names to hardware pins. IN1 and IN2 are connected to the L293D motor driver inputs and are used to select the motor direction. If IN1 = 1 and IN2 = 0, the motor rotates in one direction. If the values are reversed, the motor rotates in the opposite direction.

START_STOP_BTN on RB0 toggles the motor between running and stopped. DIR_BTN on RB1 toggles the motor direction. The code assumes active-low buttons, meaning the pin reads 0 when the button is pressed.

4. LCD Pin Definitions

```
#define LCD_RS RD2
#define LCD_EN RD3
#define LCD_D4 RD4
#define LCD_D5 RD5
#define LCD_D6 RD6
#define LCD_D7 RD7
```

The LCD is used in 4-bit mode, so only four data lines are needed: D4 to D7. LCD_RS selects whether the LCD receives a command or display data. LCD_EN is the enable signal; a pulse on this pin tells the LCD to read the data currently present on its data pins.

5. LED and Buzzer Definitions

```
#define GREEN_LED RC0
#define YELLOW_LED RC1
#define RED_LED RC3
#define BUZZER RC4
```

These outputs provide feedback to the user. The green LED indicates low speed, the yellow LED indicates medium speed, and the red LED indicates high speed. The buzzer turns on when the motor speed reaches about 95% or higher.

6. Global State Variables

```
int is_running = 0;
int current_dir = 0;
```

is_running stores whether the motor is currently running. A value of 0 means stopped, and 1 means running. current_dir stores the selected direction. A value of 0 means forward, and 1 means reverse. These variables are global because multiple functions need to read the current system state.

7. LCD Helper Functions

7.1 lcd_pulse_enable()

```
void lcd_pulse_enable(void) {  
    LCD_EN = 1;  
    __delay_us(5);  
    LCD_EN = 0;  
    __delay_us(100);  
}
```

This function creates a short pulse on the LCD enable pin. After data is placed on the LCD data pins, the enable pulse tells the LCD to read it. The small delays make sure the LCD has enough time to detect the signal correctly.

7.2 lcd_send_nibble()

```
void lcd_send_nibble(unsigned char nibble) {  
    LCD_D4 = (nibble >> 0) & 1;  
    LCD_D5 = (nibble >> 1) & 1;  
    LCD_D6 = (nibble >> 2) & 1;  
    LCD_D7 = (nibble >> 3) & 1;  
    lcd_pulse_enable();  
}
```

A nibble is 4 bits. Since the LCD is working in 4-bit mode, each 8-bit command or character must be sent in two parts. This function places each bit of the nibble on D4-D7, then calls `lcd_pulse_enable()` so the LCD reads those bits.

7.3 lcd_cmd()

```
void lcd_cmd(unsigned char cmd) {  
    LCD_RS = 0;  
    lcd_send_nibble(cmd >> 4);  
    lcd_send_nibble(cmd & 0x0F);  
    __delay_ms(2);  
}
```

This function sends a command to the LCD. It sets `LCD_RS = 0` to tell the LCD that the incoming byte is a command, not a printable character. The upper 4 bits are sent first, followed by the lower 4 bits.

7.4 lcd_data()

```
void lcd_data(unsigned char data) {  
    LCD_RS = 1;  
    lcd_send_nibble(data >> 4);  
    lcd_send_nibble(data & 0x0F);  
    __delay_us(100);  
}
```

This function sends a character to the LCD for display. It sets `LCD_RS = 1`, which means the LCD treats the byte as text data. For example, sending 'A' will display the letter A at the current cursor position.

7.5 `lcd_print()`

```
void lcd_print(const char *text) {
    while (*text) {
        lcd_data(*text++);
    }
}
```

This function prints a full string on the LCD. It loops through the text one character at a time until it reaches the null terminator at the end of the string. Each character is sent using `lcd_data()`.

7.6 `lcd_set_cursor()`

```
void lcd_set_cursor(unsigned char row, unsigned char col) {
    unsigned char address;

    if (row == 1) {
        address = 0x80 + (col - 1);
    } else {
        address = 0xC0 + (col - 1);
    }

    lcd_cmd(address);
}
```

This function moves the LCD cursor to a specific row and column. The first row starts at address `0x80`, and the second row starts at `0xC0`. The function calculates the correct address and sends it as an LCD command.

7.7 `lcd_clear()`

```
void lcd_clear(void) {
    lcd_cmd(0x01);
    __delay_ms(2);
}
```

This function clears the LCD screen. Command `0x01` tells the LCD to clear all characters and return the cursor to the beginning. The delay is needed because clearing the display takes longer than writing a normal character.

7.8 `lcd_init()`

```
void lcd_init(void) {
    __delay_ms(20);
    LCD_RS = 0;
    LCD_EN = 0;
    lcd_send_nibble(0x03);
    __delay_ms(5);
    lcd_send_nibble(0x03);
    __delay_us(150);
    lcd_send_nibble(0x03);
}
```

```
    lcd_send_nibble(0x02);  
    lcd_cmd(0x28);  
    lcd_cmd(0x0C);  
    lcd_cmd(0x06);  
    lcd_clear();  
}
```

This function initializes the LCD when the program starts. It waits for the LCD to stabilize, then sends the standard HD44780 initialization sequence for 4-bit mode. Command 0x28 selects 4-bit mode with two display lines, 0x0C turns the display on and hides the cursor, and 0x06 enables automatic cursor increment after each character.

8. ADC Functions

8.1 init_adc()

```
void init_adc(void) {  
    ADCON0 = 0x41;  
    ADCON1 = 0x8E;  
}
```

This function configures the analog-to-digital converter. `ADCON0` enables the ADC and selects channel 0, which is RA0/AN0. `ADCON1` right-justifies the 10-bit result and configures RA0 as analog while the other pins are digital.

8.2 read_adc()

```
unsigned int read_adc(void) {  
    __delay_ms(2);  
    GO_nDONE = 1;  
    while (GO_nDONE);  
    return ((ADRESH << 8) + ADRESL);  
}
```

This function reads the potentiometer value. It first waits briefly for acquisition time, then starts the ADC conversion by setting `GO_nDONE = 1`. The loop waits until conversion is complete. Finally, the function combines `ADRESH` and `ADRESL` into a 10-bit value from 0 to 1023.

9. PWM Speed Control

9.1 init_pwm()

```
void init_pwm(void) {  
    TRISC2 = 0;  
    CCP1CON = 0x0C;  
    PR2 = 0x63;  
    T2CON = 0x04;  
}
```

This function configures PWM output on RC2/CCP1. `TRISC2 = 0` makes RC2 an output. `CCP1CON = 0x0C` puts CCP1 in PWM mode. `PR2` sets the PWM period, and `T2CON = 0x04` turns Timer2 on because the PWM module depends on Timer2.

9.2 set_speed()

```
void set_speed(unsigned int duty) {  
    CCP1L = duty >> 2;  
    CCP1CON = (CCP1CON & 0xCF) | ((duty & 0x03) << 4);  
}
```

This function updates the PWM duty cycle. The ADC value is already 10 bits, so it can be mapped directly to the PWM duty cycle. The upper 8 bits are stored in CCP1L, while the lower 2 bits are placed in the correct duty-cycle bits inside CCP1CON. A larger duty cycle means a higher motor speed.

9.3 adc_to_percent()

```
unsigned char adc_to_percent(unsigned int adc_value) {  
    return (unsigned char)((((unsigned long)adc_value * 100UL) / 1023UL));  
}
```

This function converts the ADC value from the range 0-1023 into a percentage from 0 to 100. The percentage is used for LCD display and LED range selection. `unsigned long` is used during multiplication to avoid overflow.

10. display_number()

```
void display_number(unsigned char number) {  
    if (number >= 100) {  
        lcd_data('1');  
        lcd_data('0');  
        lcd_data('0');  
    } else {  
        lcd_data((number / 10) + '0');  
        lcd_data((number % 10) + '0');  
        lcd_data(' ');  
    }  
}
```

This function prints a percentage number on the LCD. If the number is 100, it prints all three digits. Otherwise, it prints the tens digit and ones digit. Adding '0' converts a numeric digit into its ASCII character. The extra space helps clear old digits from previous LCD updates.

11. update_traffic_light()

```
void update_traffic_light(unsigned char speed_percent) {
    GREEN_LED = 0;
    YELLOW_LED = 0;
    RED_LED = 0;
    BUZZER = 0;

    if (!lis_running || speed_percent == 0) {
        return;
    }

    if (speed_percent < 34) {
        GREEN_LED = 1;
    } else if (speed_percent < 67) {
        YELLOW_LED = 1;
    } else {
        RED_LED = 1;
    }

    if (speed_percent >= 95) {
        BUZZER = 1;
    }
}
```

This function updates the three LEDs and the buzzer according to motor speed. At the beginning, all outputs are turned off to clear any previous state. If the motor is stopped or the speed is 0%, the function returns immediately.

If the speed is below 34%, the green LED turns on. If it is between 34% and 66%, the yellow LED turns on. If it is 67% or higher, the red LED turns on. If the speed reaches 95% or more, the buzzer also turns on as a maximum-speed warning.

12. update_lcd()

```
void update_lcd(unsigned char speed_percent) {
    lcd_set_cursor(1, 1);
    lcd_print("Speed: ");
    display_number(speed_percent);
    lcd_print("%  ");

    lcd_set_cursor(2, 1);
    lcd_print("Dir: ");

    if (!lis_running) {
        lcd_print("Stopped ");
    } else if (current_dir == 0) {
        lcd_print("Forward ");
    } else {
        lcd_print("Reverse ");
    }
}
```


This function refreshes the LCD. The first row displays the speed percentage. The second row displays the motor state or direction. If the system is stopped, it prints Stopped. If the motor is running and `current_dir` is 0, it prints Forward. Otherwise, it prints Reverse.

13. main(): Local Variables

```
void main(void) {  
    unsigned int pot_value = 0;  
    unsigned char speed_percent = 0;
```

`pot_value` stores the raw ADC reading from the potentiometer. This value ranges from 0 to 1023.

`speed_percent` stores the same speed after conversion to a percentage from 0 to 100.

14. Port Direction Setup

```
TRISA = 0x01;  
TRISB = 0x03;  
TRISD = 0x00;  
  
TRISC0 = 0;  
TRISC1 = 0;  
TRISC3 = 0;  
TRISC4 = 0;
```

TRIS registers define whether each pin is an input or output. On PIC microcontrollers, 1 means input and 0 means output. `TRISA = 0x01` makes RA0 an input for the potentiometer. `TRISB = 0x03` makes RB0 and RB1 inputs for the buttons. `TRISD = 0x00` makes all PORTD pins outputs for motor direction and LCD control.

RC0, RC1, RC3, and RC4 are configured as outputs for the LEDs and buzzer. RC2 is configured as PWM output inside `init_pwm()`.

15. Clearing Outputs and Initializing Modules

```
PORTC = 0x00;  
PORTD = 0x00;  
  
init_adc();  
init_pwm();  
lcd_init();
```

The program clears PORTC and PORTD so all outputs start in a known off state. This prevents accidental motor movement or LED activation during startup. Then it initializes the ADC, PWM module, and LCD.

16. Startup Message

```
lcd_set_cursor(1, 1);  
lcd_print("Fan Controller");  
lcd_set_cursor(2, 1);  
lcd_print("Ready...");
```

```
__delay_ms(1000);  
lcd_clear();
```

This section displays a startup message for one second. The first row shows the project name, and the second row shows that the controller is ready. The screen is then cleared before the live speed and direction display begins.

17. The Main Infinite Loop

The `while(1)` loop is the main operating loop. It runs forever, constantly checking the buttons, reading the potentiometer, updating the motor, refreshing the LEDs and buzzer, and updating the LCD.

17.1 Start/Stop Button Handling

```
if (START_STOP_BTN == 0) {  
    __delay_ms(50);  
    if (START_STOP_BTN == 0) {  
        is_running = !is_running;  
        while (START_STOP_BTN == 0);  
    }  
}
```

Since the button is active low, the condition becomes true when the button is pressed. The 50 ms delay provides simple debouncing, which helps ignore mechanical switch noise. If the button is still pressed after the delay, `is_running` is toggled. The final loop waits until the button is released so one press does not toggle the state multiple times.

17.2 Direction Button Handling

```
if (DIR_BTN == 0) {  
    __delay_ms(50);  
    if (DIR_BTN == 0) {  
        current_dir = !current_dir;  
        while (DIR_BTN == 0);  
    }  
}
```

This block works like the start/stop button logic, but it toggles `current_dir`. If the direction was forward, it becomes reverse. If it was reverse, it becomes forward. The debounce delay and release-wait loop prevent repeated toggles from a single press.

17.3 When the Motor Is Running

```
if (is_running) {  
    pot_value = read_adc();  
    speed_percent = adc_to_percent(pot_value);  
    set_speed(pot_value);  
  
    if (current_dir == 0) {  
        IN1 = 1;  
        IN2 = 0;  
    } else {  
        IN1 = 0;  
        IN2 = 1;  
    }  
}
```

If the system is running, the program reads the potentiometer using `read_adc()`. The raw ADC value is converted into a percentage for display, and the same raw value is passed to `set_speed()` to update the PWM

duty cycle.

The program then sets the motor direction. If `current_dir` is 0, `IN1` is high and `IN2` is low, producing forward rotation. Otherwise, the outputs are reversed, producing reverse rotation.

17.4 When the Motor Is Stopped

```
} else {  
    pot_value = 0;  
    speed_percent = 0;  
    set_speed(0);  
    IN1 = 0;  
    IN2 = 0;  
}
```

If the system is not running, the speed variables are set to 0, the PWM duty cycle is set to 0, and both motor direction pins are cleared. This stops the motor and causes it to coast, depending on the motor driver wiring.

17.5 Updating Outputs and LCD

```
update_traffic_light(speed_percent);  
update_lcd(speed_percent);  
__delay_ms(150);
```

At the end of each loop cycle, the LEDs and buzzer are updated according to speed, and the LCD is refreshed with the current speed and direction. The 150 ms delay makes the display easier to read and prevents excessive refresh speed.

18. Complete Program Summary

- The program initializes ADC, PWM, LCD, and I/O pins.
- `RB0` toggles the motor between stopped and running.
- `RB1` toggles the motor direction.
- `RA0` reads the potentiometer as a 10-bit ADC value.
- `RC2` outputs PWM to control the motor speed through the L293D.
- `RD0` and `RD1` control the motor direction through the L293D inputs.
- The LCD displays speed percentage and current state.
- The LEDs indicate low, medium, and high speed.
- The buzzer turns on when the speed is close to maximum.

GitHub note: For a clean repository, upload the source code, this PDF, a README file, and any circuit diagram or simulation screenshot you have. The README can briefly describe the components, pin mapping, and build steps in MPLAB X with XC8.